



# MIPS SIMULATOR

CS3339

DeAndre Johnson & Brandon Montesinos



# Goals of this Project



## Implement a 5-stage MIPS pipeline (IF, ID, EX, MEM, WB)

- Support a specific instruction set:
  - ADD, ADDI, SUB, MUL, AND, OR, SLL, SRL, LW, SW, BEQ, J, NOP
- Decode instructions from assembly text
- Maintain register file + memory
- Print final results
- Provide debug mode showing:
  - **Pipeline registers**
  - **Control signals**
  - **Register file each cycle**

### main.cpp

- Program entry point
- Reads the .asm file
- Creates the CPU object
- Starts the simulation loop

### parser.cpp / parser.h

- Reads the assembly file
- Converts raw text into structured instruction objects
- Resolves labels for branches and jumps

### cpu.cpp / cpu.h

- Holds the CPU state:
  - Register file
  - Memory
  - Program Counter (PC)
  - Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB)
- Runs the pipeline each cycle
- Manages branching, flushing, and advancing stages

### stages.cpp / stages.h

- Implements logic for each pipeline stage:
  - IF: Instruction Fetch
  - ID: Instruction Decode & register read
  - EX: ALU operation & branch decision
  - MEM: Memory load/store
  - WB: Write results back to registers

### debug.cpp / debug.h

- Prints pipeline state each cycle in debug mode
- Prints final register file and memory contents



# main.cpp

**main.cpp** starts in `main()`, where it parses command-line arguments, sets the **debugMode** flag, and grabs the input **.asm** filename. It then opens the file with an **ifstream**, builds an **ErrorHandler** and **Parser**, and calls **parser.parse(inputFile)** to turn the raw text into a Program. If no parse errors are reported, it constructs **CPU cpu(program, debugMode)** and finally calls **cpu.run()**, so main's whole job is to validate input, set up objects, and then hand complete control to the CPU.

- This is the entry point of the entire program.
- It parses/reads the .asm file we pass on in the command line.
- Opens the assembly file with ifstream
- Prints header
- It then creates a CPU object and hands the loaded instructions to it.
- Finally, it calls CPU::run to start the simulation.

```
1
2 int main(int argc, char* argv[]) {
3     if (argc < 2) {
4         printUsage(argv[0]);
5         return 1;
6     }
7
8     string filename;
9     bool debugMode = false;
10
11     // Parse command line arguments
12     for (int i = 1; i < argc; i++) {
13         string arg = argv[i];
14
15         if (arg == "-debug" || arg == "-d") {
16             debugMode = true;
17         } else if (arg == "-help" || arg == "-h") {
18             printUsage(argv[0]);
19             return 1;
20         } else if (arg.empty() && arg[i] == '-') {
21             cerr << "Unknown option: " << arg << endl;
22             printUsage(argv[0]);
23             return 1;
24         } else {
25             filename = arg;
26         }
27     }
28
29     if (filename.empty()) {
30         cerr << "Error: No input file specified" << endl;
31         printUsage(argv[0]);
32         return 1;
33     }
34
35     // Open input file
36     ifstream inputFile(filename);
37     if (!inputFile) {
38         cerr << "Error: Could not open file " << filename << " " << endl;
39         return 1;
40     }
41
42     cout << "=== MIPS PIPELINE SIMULATOR ===" << endl;
43     cout << "CS3339 Fall 2025" << endl;
44     cout << "Input file: " << filename << endl;
45     if (debugMode) {
46         cout << "Debug mode: ENABLED" << endl;
47     }
48
49     // Parse the assembly file
50     ErrorHandler errorHandler;
51     Parser parser(errorHandler);
52
53     Program program = parser.parse(inputFile);
54     inputFile.close();
55
56     // Check for parsing errors
57     if (errorHandler.hasErrors()) {
58         errorHandler.printErrors();
59     }
60
61     // Generate error file name from input file
62     string errorFile = filename;
63     size_t dotPos = errorFile.rfind('.');
64     if (dotPos != string::npos) {
65         errorFile = errorFile.substr(0, dotPos);
66     }
67     errorFile += ".err";
68     errorHandler.writeToFile(errorFile);
69     return 1;
70 }
71
72 // Check if we have any instructions
73 if (program.instructions.empty()) {
74     cerr << "Error: No instructions found in input file" << endl;
75     return 1;
76 }
77
78 cout << "Instructions loaded: " << program.instructions.size() << endl;
79
80 // Create and run the CPU simulator
81 try {
82     CPU cpu(program, debugMode);
83     cpu.run();
84
85     cout << endl << "=== SIMULATION COMPLETE ===" << endl;
86 } catch (const exception& e) {
87     cerr << endl << "Runtime Error: " << e.what() << endl;
88     return 1;
89 }
```



# parser.cpp / parser.h

In **Parser::parse(istream& input)**, the code first reads the file line by line into a local **LineInfo** vector, stripping comments and whitespace and recording labels into **program.labels** by mapping label names to their instruction indices.

On the second pass, it walks that **instrLines** vector, extracts the mnemonic with an **istream**, converts it to an Opcode via **parseOpcode**, and splits the rest with **splitOperands**. For each opcode it fills an Instruction struct (registers via **parseRegister**, immediates/targets via helpers, and branch/jump labels resolved through the label map), then pushes it into **program.instructions**, so by the end, Program holds a clean, fully decoded representation of the original assembly.

```
1 Program Parser::parse(istream& input) {
2     program.instructions.clear();
3     program.labels.clear();
4
5     // Store raw lines with their instruction text
6     struct LineInfo {
7         string text;
8         int srcLine;
9     };
10    vector<LineInfo> instrLines;
11
12    string line;
13    int srcLineNum = 0;
14    size_t instrIndex = 0;
15
16    // First pass: collect all instruction lines and labels
17    while (getline(input, line)) {
18        srcLineNum++;
19
20        // Remove comments
21        size_t commentPos = line.find('#');
22        if (commentPos != string::npos) {
23            line = line.substr(0, commentPos);
24        }
25
26        line = trim(line);
27        if (line.empty()) continue;
28
29        // Check for label
30        size_t colonPos = line.find(':');
31        if (colonPos != string::npos) {
32            string label = trim(line.substr(0, colonPos));
33            if (!label.empty()) {
34                if (program.labels.count(label)) {
35                    errorHandler.addError(srcLineNum, "Duplicate label: " + label);
36                } else {
37                    // Label points to next instruction index
38                    program.labels[label] = instrIndex;
39                }
40            }
41            line = trim(line.substr(colonPos + 1));
42            if (line.empty()) continue;
43        }
44
45        // This is an instruction line
46        instrLines.push_back({line, srcLineNum});
47        instrIndex++;
48    }
```



```
1 // Parse assembly from input stream
2 Program parse(std::istream& input);
```

# cpu.cpp / cpu.h

The **CPU** constructor sets pc and cycleCount to 0, stores the **debugMode** flag, zero-fills the **registers array**, allocates **memory** with 1024 zeros, and resets all pipeline registers (**if\_id**, **id\_ex**, **ex\_mem**, **mem\_wb**) to their default “invalid” state. **CPU::run()** prints a binary listing using **Debug::printBinaryRepresentation**, then enters a loop that continues while pc is within the instruction range or any pipeline register is still valid, counting cycles and calling **stepPipeline()** each time. Inside **stepPipeline()**, the CPU calls **PipelineStages::wbStage**, **memStage**, **exStage**, and **idStage** in reverse order, uses **branchTaken/branchTarget** from EX to optionally flush **IF\_ID** and **ID\_EX** and redirect pc, updates the pipeline registers to their “next” versions, and finally enforces **registers[0] = 0** to keep **\$zero** correct.

- Created a register file of 32 registers
- Created 1024 words of memory (all zeros)
- Set program counter (PC) to 0
- Creates Four empty pipeline registers:
  - IF/ID
  - ID/EX
  - EX/MEM
  - MEM/WB

```
1 // Parsed instruction
2 struct Instruction {
3     Opcode op;
4     int rs; // Source register 1
5     int rt; // Source register 2 / Destination for I-type
6     int rd; // Destination register for R-type
7     int shamt; // Shift amount
8     int32_t imm; // Immediate value
9     size_t target; // Jump/branch target (instruction index)
10    std::string text; // Original text for debug
11
12    Instruction() : op(Opcode::NOP), rs(0), rt(0), rd(0), shamt(0), imm(0), target(0) {}
13 };
14
15 // Control signals generated during decode
16 struct ControlSignals {
17     bool regDst; // 1-rd (R-type), 0-rt (I-type)
18     bool aluSrc; // 1-immediate, 0-register
19     bool memToReg; // 1-memory data, 0-ALU result
20     bool regWrite; // Write to register file
21     bool memRead; // Read from memory
22     bool memWrite; // Write to memory
23     bool branch; // Branch instruction
24     bool jump; // Jump instruction
25
26     ControlSignals() : regDst(false), aluSrc(false), memToReg(false),
27                       regWrite(false), memRead(false), memWrite(false),
28                       branch(false), jump(false) {}
29 };
30
31 // IF/ID pipeline register
32 struct IF_ID {
33     bool valid;
34     size_t pc;
35     Instruction instr;
36
37     IF_ID() : valid(false), pc(0) {}
38 };
39
40 // ID/EX pipeline register
41 struct ID_EX {
42     bool valid;
43     size_t pc;
44     Instruction instr;
45     ControlSignals ctrl;
46     int32_t rsVal; // Value read from rs
47     int32_t rtVal; // Value read from rt
48     int32_t signtImm
```

```
1 // The CPU class that runs the simulation
2 class CPU {
3 private:
4     // Program
5     std::vector<Instruction> instructions;
6
7     // Architectural state
8     size_t pc;
9     std::array<int32_t, 32> registers;
10    std::vector<int32_t> memory;
11
12    // Pipeline registers
13    IF_ID if_id;
14    ID_EX id_ex;
15    EX_MEM ex_mem;
16    MEM_WB mem_wb;
17
18    // Statistics
19    size_t cycleCount;
20    bool debugMode;
21
22    // Helper methods
23    ControlSignals generateControl(const Instruction& instr);
24    int32_t executeALU(const Instruction& instr, int32_t op1, int32_t op2);
25    bool pipelineEmpty() const;
```

# stages.cpp / stages.h

The pipeline stages are implemented as individual functions that each operate on their corresponding pipeline register:

- IF fetches the next instruction using PC
- ID decodes the instruction and reads registers
- EX performs ALU work and branch decisions
- MEM handles loads and stores
- WB writes results back to the register file

During execution, **cpu.run()** repeatedly calls **stepPipeline()**, which runs these stages in order each cycle and transfers data from one pipeline register to the next. The run loop continues as long as instructions remain or the pipeline isn't empty, printing the full pipeline state each cycle when **--debug** is enabled. Once execution finishes, the CPU outputs the total cycle count along with final register and memory state.

```
1 ControlSignals PipelineStages::generateControl(const Instruction& instr) {
2     ControlSignals ctrl;
3
4     switch (instr.op) {
5     case Opcode::ADD:
6     case Opcode::SUB:
7     case Opcode::MUL:
8     case Opcode::AND:
9     case Opcode::OR:
10        ctrl.regDst = true; // Write to rd
11        ctrl.aluSrc = false; // Use register
12        ctrl.regWrite = true; // Write result
13        break;
14
15    case Opcode::SLT:
16    case Opcode::SLL:
17        ctrl.regDst = true; // Write to rd
18        ctrl.aluSrc = false; // rt value goes through
19        ctrl.regWrite = true;
20        break;
21
22    case Opcode::ADDI:
23        ctrl.regDst = false; // Write to rt
24        ctrl.aluSrc = true; // Use immediate
25        ctrl.regWrite = true;
26        break;
27
28    case Opcode::LW:
29        ctrl.regDst = false; // Write to rt
30        ctrl.aluSrc = true; // Use immediate for offset
31        ctrl.memToReg = true; // Data from memory
32        ctrl.memRead = true;
33        ctrl.regWrite = true;
34        break;
35
36    case Opcode::SW:
37        ctrl.aluSrc = true; // Use immediate for offset
38        ctrl.memWrite = true;
39        break;
40
41    case Opcode::BEQ:
42        ctrl.branch = true;
43        break;
44
45    case Opcode::J:
46        ctrl.jump = true;
47        break;
48
49    case Opcode::NOP:
50    default:
51        // All signals false (default)
52        break;
53    }
54
55    return ctrl;
56 }
```

# The 5 STAGES

## 1. IF (Instruction Fetch)

- Reads the next instruction using the PC
- Places it into the IF/ID pipeline register
- Updates PC for the next instruction

## 2. ID (Instruction Decode & Register Read)

- Reads instruction from IF/ID
- Extracts opcode, registers, immediate
- Reads the register file (registers[rs], registers[rt])
- Generates the control signals needed by later stages
- Writes all of this into ID/EX

## 3. EX (Execute / ALU)

- Performs arithmetic and logic: ADD, SUB, MUL, AND, OR, shifts
- Calculates addresses for LW/SW
- Evaluates BEQ (branch taken or not) and Handles J (jump)
- Passes results to EX/MEM
- If a branch/jump is taken, CPU flushes incorrect instructions and updates PC

## 4. MEM (Memory access)

- LW (Load Word): fetch a 32-bit value from memory
- SW (Store Word): save a 32-bit value into memory
- Uses the address computed in EX
- Writes result into MEM/WB

## 5. WB (Write Back)

- If regWrite is set, the result is written to the destination register
- If the instruction was LW, the value comes from memory
- Otherwise, it comes from the ALU

# debug.cpp / debug.h

The debug system is implemented through the **Debug** class, which exposes print functions that mirror the CPU's internal state. When **--debug** is enabled, the CPU calls **Debug::printPipelineState** every cycle, which prints the program counter, all four pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB), the active control signals generated during decode, and the entire register file. These functions allow you to trace execution step-by-step, making it easy to verify that pipeline flow, branching, stalling, and data movement match what the MIPS pipeline is supposed to do.

```
1 void Debug::printPipelineState(const CPU& cpu) {
2     std::cout << "\n===== CYCLE " << cpu.getCycleCount() << " =====\n";
3     std::cout << "PC = " << cpu.getPC() << "\n\n";
4
5     std::cout << "--- Pipeline Registers ---\n";
6     printIF_ID(cpu.getIF_ID());
7     printID_EX(cpu.getID_EX());
8     printEX_MEM(cpu.getEX_MEM());
9     printMEM_WB(cpu.getMEM_WB());
10
11     printRegisters(cpu.getRegisters());
12     std::cout << "=====\n";
13 }
```

```
1 void Debug::printRegisters(const std::array<int32_t, 32>& registers) {
2     std::cout << "\n--- Register File ---\n";
3     bool found = false;
4
5     for (int i = 0; i < 32; i++) {
6         if (registers[i] != 0) {
7             std::cout << std::setw(6) << regName(i) << " ($" << std::setw(2) << i << "): "
8                 << std::setw(11) << registers[i];
9
10            // Print hex value
11            std::cout << " (0x" << std::hex << std::setfill('0')
12                << std::setw(8) << static_cast<uint32_t>(registers[i])
13                << std::dec << std::setfill(' ') << ")\n";
14            found = true;
15        }
16    }
17
18    if (!found) {
19        std::cout << " (all registers are zero)\n";
20    }
21 }
22
23 void Debug::printMemory(const std::vector<int32_t>& memory) {
24     std::cout << "\n--- Memory (non-zero) ---\n";
25     bool found = false;
26
27     for (size_t i = 0; i < memory.size(); i++) {
28         if (memory[i] != 0) {
29             std::cout << " [" << std::setw(4) << (i * 4) << "]: "
30                 << std::setw(11) << memory[i]
31                 << " (0x" << std::hex << std::setfill('0')
32                 << std::setw(8) << static_cast<uint32_t>(memory[i])
33                 << std::dec << std::setfill(' ') << ")\n";
34             found = true;
35         }
36     }
37
38    if (!found) {
39        std::cout << " (all memory locations are zero)\n";
40    }
41 }
42
43 void Debug::printControlSignals(const ControlSignals& ctrl) {
44     std::cout << " RegDst=" << ctrl.regDst
45         << " ALUSrc=" << ctrl.aluSrc
46         << " MemToReg=" << ctrl.memToReg
47         << " RegWrite=" << ctrl.regWrite
48         << " MemRead=" << ctrl.memRead
49         << " MemWrite=" << ctrl.memWrite
50         << " Branch=" << ctrl.branch
51         << " Jump=" << ctrl.jump << "\n";
52 }
53
54 void Debug::printIF_ID(const IF_ID& reg) {
55     std::cout << " IF/ID: ";
56     if (reg.valid) {
57         std::cout << "[" << reg.instr.text << "]\n";
58         std::cout << " PC=" << reg.pc << "\n";
59     } else {
60         std::cout << "[empty]\n";
61     }
62 }
```

# Pipeline Execution

## Running the Simulation

### Step 1 — Print Binary Representation

- Converts each instruction into a 32-bit machine code value
- Simulation has not started yet — this is only informational output

### Step 2 — Main Simulation Loop

- Continues while instructions remain **or** the pipeline has not fully drained
- Each loop iteration = one clock cycle
- Calls stepPipeline() to advance all five pipeline stages
- If --debug is enabled, prints pipeline registers, PC, control signals, and the register file each cycle

### Step 3 — Simulation Finishes

- Prints total cycle count
- Displays the final register file
- Shows all non-zero memory locations

```
1 void CPU::stepPipeline() {
2     // Next cycle values
3     IF_ID next_if_id;
4     ID_EX next_id_ex;
5     EX_MEM next_ex_mem;
6     MEM_WB next_mem_wb;
7
8     // == WB Stage ==
9     PipelineStages::wbStage(mem_wb, registers);
10
11     // == MEM Stage ==
12     next_mem_wb = PipelineStages::memStage(ex_mem, memory);
13
14     // == EX Stage ==
15     bool branchTaken = false;
16     size_t branchTarget = 0;
17     next_ex_mem = PipelineStages::exStage(id_ex, branchTaken, branchTarget);
18
19     // == ID Stage ==
20     next_id_ex = PipelineStages::idStage(if_id, registers);
21
22     // == IF Stage ==
23     if (pc < instructions.size()) {
24         next_if_id.valid = true;
25         next_if_id.pc = pc;
26         next_if_id.instr = instructions[pc];
27         pc++;
28     }
29
30     // == Branch / Jump handling ==
31     if (branchTaken) {
32         next_if_id = IF_ID(); // flush
33         next_id_ex = ID_EX(); // flush
34         pc = branchTarget; // redirect PC
35     }
36
37     // Update pipeline registers
38     mem_wb = next_mem_wb;
39     ex_mem = next_ex_mem;
40     id_ex = next_id_ex;
41     if_id = next_if_id;
42
43     // Enforce $zero = 0
44     registers[0] = 0;
45 }
46
47 void CPU::run() {
48     cout << "\n=== STARTING SIMULATION ===" << endl;
49
50     Debug::printBinaryRepresentation(instructions);
51
52     const size_t MAX_CYCLES = 10000;
53
54     while ((pc < instructions.size() || !pipelineEmpty()) && cycleCount < MAX_CYCLES) {
55         cycleCount++;
56         stepPipeline();
57
58         if (debugMode) {
59             Debug::printPipelineState(*this);
60         }
61     }
62
63     if (cycleCount >= MAX_CYCLES) {
64         cerr << "Warning: Simulation stopped after " << MAX_CYCLES
65              << " cycles" << endl;
66     }
67
68     cout << "\n=== FINAL MACHINE STATE ===" << endl;
69     cout << "Total Cycles: " << cycleCount << endl;
70     Debug::printRegisters(registers);
71     Debug::printMemory(memory);
72 }
```

# Time Benchmark

## DeAndre code time:

1. Real .010s  
User 0s  
Sys .005s
2. Real .009s  
User .002s  
Sys .002s
3. Real .010s  
User 0s  
Sys .005s

## Brandon code time:

1. Real 8.004s  
User 0.001s  
Sys 0.013s
2. Real 7.624s  
User 0.000s  
Sys 0.013s
3. Real 9.264s  
User 0.001s  
Sys 0.007s

**Brandon compared DeAndre's code and ran the tests 3 times. DeAndre's code is much faster compare to Brandon's, by 8.297 seconds.**

# Finishing the Simulation

The simulation continues running until two conditions are met:

1. **The Program Counter moves past the last instruction**, and
2. **All pipeline registers become empty**, meaning the pipeline has fully drained.

At this point, no more instructions are executing or waiting in the pipeline.

Final Machine State (Summary):

- **Total Cycles: 76**
- **Instructions Loaded: 72**
- **Registers Modified: 15**
- **Memory Writes: 2 (addresses 0 and 4)**

```
1  === MIPS PIPELINE SIMULATOR ===
2  CS3339 Fall 2025
3  Input file: tests/test.asm
4  Instructions loaded: 72
5
6  === STARTING SIMULATION ===
7
8  --- Binary Representation ---
9  Addr      Binary      Assembly
10 -----
11 0  00100000000010000000000000001010  ADDI $t0, $zero, 10
12 4  00000000000000000000000000000000  NOP
13 8  00000000000000000000000000000000  NOP
14 12 00000000000000000000000000000000  NOP
15 16 0010000000001001000000000000101  ADDI $t1, $zero, 5
16 20 00000000000000000000000000000000  NOP
17 24 00000000000000000000000000000000  NOP
18 28 00000000000000000000000000000000  NOP
19 32 000000010000100101000000100000  ADD $t2, $t0, $t1
20 36 00000000000000000000000000000000  NOP
21 40 00000000000000000000000000000000  NOP
22 44 00000000000000000000000000000000  NOP
23 48 0000000100001001010110000010010  SUB $t3, $t0, $t1
24 52 00000000000000000000000000000000  NOP
25 56 00000000000000000000000000000000  NOP
26 60 00000000000000000000000000000000  NOP
27 64 011000100001001011000000000010  MUL $t4, $t0, $t1
28 68 00000000000000000000000000000000  NOP
29 72 00000000000000000000000000000000  NOP
30 76 00000000000000000000000000000000  NOP
```